

ui.restructured_text 全面详解

`ui.restructured_text` 是 NiceGUI 中用于渲染 reStructuredText (reST) 格式文档的专用组件，基于 `docutils` 解析器实现，支持 reST 标准语法及扩展功能（如代码块、表格、数学公式、交叉引用等）。该组件继承自 `ui.element` 基类，具备样式自定义、动态更新、数据绑定等核心能力，适用于技术文档、API 说明、学术文档等场景，是 reST 格式内容在 Web 界面中展示的核心工具。

一、核心定位与初始化参数

1. 核心定位

- reST 文档渲染：专注于解析和渲染 reStructuredText 格式文本，兼容 reST 标准语法及 `docutils` 扩展；
- 技术文档适配：天然支持代码块高亮、表格、公式、脚注等技术文档常用元素，无需额外配置；
- 继承基础能力：作为 `ui.element` 子类，支持样式自定义、动态更新、数据绑定等通用功能。

2. 初始化参数

参数名	类型与说明	默认值	关键注意事项
content	待渲染的 reST 文本（支持单行字符串或多行字符串）	-	核心必填参数，需遵循 reST 语法规则，支持 <code>docutils</code> 兼容的扩展语法
settings	<code>docutils</code> 配置字典（如 <code>{'initial_header_level': 2}</code> ）	<code>{}</code>	用于配置解析器行为（如标题级别、代码高亮、图片处理等），需符合 <code>docutils</code> 规范
继承参数			继承自 <code>ui.element</code> 的 <code>classes</code> 、 <code>style</code> 、 <code>html_id</code> 、 <code>visible</code> 等属性

基础使用示例

```
from nicegui import ui

# 1. 简单 reST 渲染（标准语法）
ui.restructured_text('''
=====
Hello reST!
=====

这是 **加粗** 文本，这是 *斜体* 文本，这是 ``行内代码``。

列表示例
-----
- 无序列表项 1
- 无序列表项 2
  - 嵌套列表项
- 无序列表项 3
''')
```

```
有序列表
-----

1. 有序列表项 1
2. 有序列表项 2
3. 有序列表项 3
'''

# 2. 带配置的 reST 渲染（指定初始标题级别）
ui.restructured_text('''
标题 1（实际渲染为 h2）
=====

标题 2（实际渲染为 h3）
-----

''', settings={'initial_header_level': 2}) # 初始标题级别从 h2 开始

ui.run()
```

二、核心功能与使用场景

1. 标题与层级结构

reST 标题通过下划线字符（如 `=`、`-`、`~` 等）定义层级，`ui.restructured_text` 会自动解析为 HTML 标题标签（`h1-h6`），支持通过 `settings` 调整初始标题级别。

语法示例 (reST)	渲染结果 (HTML)	说明
一级标题 <code>``=====</code>	<code><h1>一级标题</code> <code></h1></code>	下划线长度需 ≥ 标题长度，常用 <code>=</code> 表示最高级标题
二级标题 <code>``-----</code> <code>-</code>	<code><h2>二级标题</code> <code></h2></code>	常用 <code>-</code> 表示二级标题，下划线字符可自定义（需保持一致）
三级标题 <code>``~::~::~</code>	<code><h3>三级标题</code> <code></h3></code>	常用 <code>~</code> 表示三级标题，层级顺序由下划线字符出现顺序决定

示例：

```
from nicegui import ui

ui.restructured_text('''
文档标题（h1）
=====

章节 1（h2）
-----

这是章节 1 的内容。

子章节 1.1（h3）
~::~::~
```

这是子章节 1.1 的内容。

子章节 1.2 (h3)

~~~~~

这是子章节 1.2 的内容。

章节 2 (h2)

-----

这是章节 2 的内容。

''')

ui.run()

## 2. 代码块与语法高亮

支持两种代码块语法：**简单代码块**（缩进 4 个空格）和 **高亮代码块**（通过 `.. code-block::` 指令指定语言），依赖 `Pygments` 库实现语法高亮（内置依赖，无需额外安装）。

### 支持的语言

Python、JavaScript、Java、HTML、CSS、C++、Go 等（完整列表见 [Pygments 官方文档](#)）。

示例：

```
from nicegui import ui
```

```
ui.restructured_text('''
```

代码块示例

=====

1. 简单代码块（无语言指定，默认无高亮）  
缩进 4 个空格即可：

```
def greet(name):  
    return f"Hello, {name}!"
```

2. 高亮代码块（指定语言 Python）  
`.. code-block:: python`

```
from nicegui import ui  
  
def add(a: int, b: int) -> int:  
    """两数相加"""  
    return a + b  
  
ui.label(f"2 + 3 = {add(2, 3)}")  
ui.run()
```

3. 其他语言代码块（JavaScript）  
`.. code-block:: javascript`

```
const multiply = (x, y) => x * y;  
console.log(multiply(4, 5)); // 输出 20
```

```
'''  
  
ui.run()
```

### 3. 表格渲染

支持 reST 标准表格语法（简单表格、网格表格），自动解析为 HTML 表格，适配响应式布局。

#### (1) 简单表格（适用于少量数据）

```
from nicegui import ui  
  
ui.restructured_text('''  
简单表格示例  
=====
```

| 姓名 | 年龄 | 职业   |
|----|----|------|
| 张三 | 28 | 工程师  |
| 李四 | 35 | 设计师  |
| 王五 | 42 | 产品经理 |

```
''')  
  
ui.run()
```

#### (2) 网格表格（适用于复杂数据，支持单元格合并）

```
from nicegui import ui  
  
ui.restructured_text('''  
网格表格示例（支持合并单元格）  
=====
```

| 表头 1  | 表头 2  | 表头 3  |
|-------|-------|-------|
| 合并行   | 单元格 2 | 单元格 3 |
|       | 单元格 4 | 单元格 5 |
| 单元格 6 | 合并列   |       |

```
''')  
  
ui.run()
```

## 4. 数学公式渲染

支持通过 `.. math::` 指令渲染 LaTeX 公式（行内公式、块级公式），需安装依赖包：

```
pip install docutils>=0.18 sphinx # sphinx 提供公式渲染支持
```

示例：

```
from nicegui import ui

ui.restructured_text('''
数学公式示例
=====

1. 行内公式（嵌入文本中）
   勾股定理：:math:`a^2 + b^2 = c^2`，其中 :math:`c` 为斜边。

2. 块级公式（独立成行）
   .. math::
      \hat{f}(\xi) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i \xi x} dx

3. 多行公式
   .. math::
      \begin{cases}
        x + y = 5 \\
        2x - y = 1
      \end{cases}
''')

ui.run()
```

## 5. 链接与交叉引用

支持外部链接、内部锚点引用、脚注引用等，满足文档导航需求。

示例：

```
from nicegui import ui

ui.restructured_text('''
链接与引用示例
=====

1. 外部链接
   - 官方文档：`NiceGUI` 官网 <https://nicegui.io>`_
   - reST 规范：`reStructuredText` 指南
     <https://docutils.sourceforge.io/docs/user/rst/quickref.html>`_

2. 内部锚点引用（需定义锚点）
   .. _锚点-章节1:

   章节 1: 锚点目标
''')
```

-----

这是锚点目标内容。

跳转至 `章节 1`\_（通过锚点引用）。

### 3. 脚注引用

这是一段带脚注的文本 [#note1]\_。

这是另一段带脚注的文本 [#note2]\_。

```
.. rubric:: 脚注
.. [#note1] 脚注 1 的内容。
.. [#note2] 脚注 2 的内容，支持多行文本和 **格式化**。
```

```
ui.run()
```

## 6. 图片与媒体

支持通过 `.. image::` 指令嵌入图片，支持本地图片、网络图片，可配置尺寸、对齐方式。

示例：

```
from nicegui import ui

ui.restructured_text('''
图片嵌入示例
=====

1. 网络图片（指定宽度）
.. image:: https://picsum.photos/id/237/800/600
   :width: 400
   :alt: 示例图片（狗）
   :align: center

2. 本地图片（需放在项目根目录或指定路径）
.. image:: ./assets/logo.png
   :width: 200
   :alt: 本地Logo
   :align: left

3. 图片说明
.. figure:: https://picsum.photos/id/1005/800/600
   :width: 300
   :align: right

   图片说明：这是一张风景图。
''')

ui.run()
```

## 7. 动态更新内容

支持通过 `content` 属性或 `set_content` 方法动态修改 reST 内容，结合其他组件实现交互性文档展示。

示例：

```
from nicegui import ui

# 创建组件并保存引用
rst = ui.restructured_text(''')
# 初始内容
点击下方按钮切换文档内容...
''')

# 切换到「代码示例」
def show_code():
    rst.set_content(''')
    代码示例文档
    =====

    .. code-block:: python

        def dynamic_update():
            """动态更新 reST 内容示例"""
            return "Hello, Dynamic reST!"

    print(dynamic_update())
''')

# 切换到「表格示例」
def show_table():
    rst.set_content(''')
    动态表格示例
    =====

    | 功能          | 支持状态 |
    |-----|-----|
    | 动态更新      | ☒ 支持 |
    | 代码高亮      | ☒ 支持 |
    | 表格渲染      | ☒ 支持 |
    | 公式渲染      | ☒ 支持 |
    ''')

# 按钮布局
with ui.row():
    ui.button('显示代码示例', on_click=show_code)
    ui.button('显示表格示例', on_click=show_table)

ui.run()
```

## 8. 样式自定义

`ui.restructured_text` 渲染的内容会自动添加 `nicegui-restructured-text` CSS 类，可通过全局 CSS 调整内部元素样式；也可通过 `classes` 和 `style` 属性调整组件整体样式。

示例：

```
from nicegui import ui

# 1. 自定义内部元素样式（全局 CSS）
ui.add_css('''
/* 标题样式：深蓝色、底部边框 */
.nicegui-restructured-text h2 {
    color: #1e40af;
    border-bottom: 2px solid #e2e8f0;
    padding-bottom: 0.3em;
    margin-top: 1.5em;
}
/* 链接样式：橙色、无下划线 */
.nicegui-restructured-text a {
    color: #ff7e00;
    text-decoration: none;
}
.nicegui-restructured-text a:hover {
    text-decoration: underline;
}
/* 代码块样式：深色背景、圆角、内边距 */
.nicegui-restructured-text pre {
    background-color: #1f2937;
    color: #f3f4f6;
    border-radius: 0.5rem;
    padding: 1rem;
    overflow-x: auto;
}
/* 表格样式：边框、居中对齐 */
.nicegui-restructured-text table {
    border-collapse: collapse;
    width: 100%;
    margin: 1em 0;
}
.nicegui-restructured-text th, td {
    border: 1px solid #d1d5db;
    padding: 0.75em;
    text-align: center;
}
''')

# 2. 调整组件整体样式（内边距、背景、阴影）
ui.restructured_text('''
## 自定义样式示例
### 1. 链接
`NiceGUI` 官网 <https://nicegui.io>`_
```



```
### 2. 代码块
.. code-block:: python

    print("Custom style for code block!")

### 3. 表格
| 列 1 | 列 2 | 列 3 |
|-----|-----|-----|
| A     | B     | C     |
| D     | E     | F     |
'''
).classes('p-6 bg-gray-50 rounded-lg shadow-sm')

ui.run()
```

### 三、组件属性

`ui.restructured_text` 继承自 `ui.element`，除基础属性外，新增 `content` 和 `settings` 绑定属性，支持动态同步配置：

| 属性名      | 类型               | 说明                                          | 适用场景               |
|----------|------------------|---------------------------------------------|--------------------|
| content  | BindableProperty | reST 内容（可绑定到其他对象属性，支持双向同步）                  | 动态内容更新、状态同步        |
| settings | BindableProperty | <code>docutils</code> 配置字典（可绑定，支持动态调整解析器行为） | 动态切换解析规则（如标题级别）    |
| classes  | Classes[Self]    | 组件的 CSS 类（Tailwind/Quasar 类），用于整体样式调整       | 组件布局、背景、边距配置       |
| style    | Style[Self]      | 内联 CSS 样式，用于精细化样式调整                         | 字体大小、颜色、对齐方式       |
| html_id  | str              | HTML DOM 元素 ID（NiceGUI 2.16.0+ 新增）          | 原生 JS 交互、CSS 选择器定位 |
| visible  | BindableProperty | 组件可见性（布尔值，支持绑定）                             | 条件显示 / 隐藏文档内容      |

### 属性使用示例（配置绑定）

```
from nicegui import ui

# 定义状态类
class AppState:
    def __init__(self):
        self.rst_content = "# 初始标题"
        self.rst_settings = {'initial_header_level': 1} # 初始标题级别 h1

state = AppState()

# 组件属性绑定
```

```
rst = ui.restructured_text('') \
    .bind_content(state, 'rst_content') \
    .bind_settings(state, 'rst_settings')

# 滑块控制初始标题级别
slider = ui.slider(min=1, max=3, value=1, label='初始标题级别')
slider.bind_value_to(
    state, 'rst_settings',
    forward=lambda level: {'initial_header_level': level} # 同步滑块值到 settings
)

# 输入框修改内容
ui.input(
    label="修改标题",
    value=state.rst_content,
    on_change=lambda e: setattr(state, 'rst_content', e.value)
).classes('w-full mt-4')

ui.run()
```

## 四、核心方法

除继承 `ui.element` 的所有方法（如 `move`、`delete`、`tooltip` 等）外，`ui.restructured_text` 新增专属方法用于内容和配置操作：

| 方法名                                 | 参数与说明                                                                                                      | 功能描述                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------|
| <code>set_content(content)</code>   | <code>content</code> : 新的 reST 文本                                                                          | 手动设置组件内容（动态更新）           |
| <code>set_settings(settings)</code> | <code>settings</code> : 新的 <code>docutils</code> 配置字典                                                      | 手动设置解析器配置（动态调整解析规则）      |
| <code>bind_content</code>           | <code>target_object</code> : 目标对象； <code>target_name</code> : 属性名；<br><code>forward/backward</code> : 转换函数 | 内容双向绑定到目标对象属性（组件 ↔ 目标对象） |
| <code>bind_content_from</code>      | <code>target_object</code> : 目标对象； <code>target_name</code> : 属性名；<br><code>backward</code> : 转换函数         | 内容单向绑定（目标对象 → 组件）        |
| <code>bind_settings</code>          | <code>target_object</code> : 目标对象； <code>target_name</code> : 属性名；<br><code>forward/backward</code> : 转换函数 | 配置双向绑定到目标对象属性（组件 ↔ 目标对象） |
| <code>bind_settings_from</code>     | <code>target_object</code> : 目标对象； <code>target_name</code> : 属性名；<br><code>backward</code> : 转换函数         | 配置单向绑定（目标对象 → 组件）        |

### 方法使用示例（配置单向绑定）

```
from nicegui import ui

# 开关控制是否启用代码高亮
highlight_switch = ui.switch(value=True, label='启用代码高亮')
```

```
# reST 内容配置单向绑定到开关状态
rst = ui.restructured_text('''
代码高亮控制示例
=====

.. code-block:: python

    def hello():
        return "Hello, Code Highlight!"
''').bind_settings_from(
    highlight_switch, 'value',
    backward=lambda enabled: {'syntax_highlight': 'short' if enabled else None}
    # enabled=True 时启用高亮（short 表示简短高亮样式），否则禁用
)

ui.run()
```

## 五、版本兼容性与依赖说明

| 功能 / 属性          | 最低版本要求 | 依赖说明                                              |
|------------------|--------|---------------------------------------------------|
| 基础 reST 渲染       | -      | 内置 docutils 依赖（需 ≥0.17），默认安装 NiceGUI 时自动附带        |
| html_id 属性       | 2.16.0 | 新增 HTML DOM 元素 ID 配置能力                            |
| 数学公式渲染           | -      | 需安装 sphinx（pip install sphinx），sphinx 依赖 docutils |
| 代码块高亮            | -      | 内置 Pygments 依赖，支持多数编程语言                           |
| bind_settings 方法 | 3.0.0  | 新增配置绑定方法，支持 strict 参数校验目标属性（3.0.0+）               |

## 依赖安装命令

```
# 基础依赖（默认已安装）
pip install docutils>=0.17

# 数学公式渲染依赖
pip install sphinx

# 确保 NiceGUI 版本兼容（建议 ≥3.0.0 以支持完整绑定功能）
pip install nicegui>=3.0.0
```

## 六、常见问题与注意事项

- 语法规则失败：**reST 语法对缩进和格式要求严格（如标题下划线长度、列表缩进、代码块缩进），需确保内容符合 [reST 规范](#)；
- 公式渲染失败：**需安装 sphinx 依赖，且公式语法需严格遵循 LaTeX 规范（如转义字符 \\、花括号 {}）

等)；

3. **代码块无高亮**：检查是否通过 `.. code-block:: 语言名` 指定语言，且语言名正确（如 `python` 而非 `Python`）；

4. **样式自定义冲突**：`nicegui-restructured-text` 类是组件内部样式的统一前缀，自定义 CSS 时需通过该类定位内部元素，避免影响全局样式；

5. **大文档性能优化**：渲染超长 reST 文档（如万字以上）时，建议分页加载或异步渲染，避免界面卡顿；

6. **本地图片路径问题**：嵌入本地图片时，需确保图片路径相对于项目根目录，或使用绝对路径，且服务端可访问该路径。

## 七、高级应用场景示例

### 1. 本地 reST 文件阅读器

```
from nicegui import ui
import os

# 读取本地 reST 文件
def read_rst_file(file_path: str) -> str:
    if not os.path.exists(file_path):
        return "文件不存在！"
    with open(file_path, 'r', encoding='utf-8') as f:
        return f.read()

# 示例：创建临时 reST 文件
with open('example.rst', 'w', encoding='utf-8') as f:
    f.write('''
本地 reST 文件示例
=====

这是一个本地 reST 文件的内容，包含：

- 标题层级
- 代码块
- 表格

.. code-block:: python

    # 读取本地文件示例
    def read_file(path):
        with open(path, 'r') as f:
            return f.read()

表格示例
-----
| 文件名      | 大小   | 类型     |
|-----|-----|-----|
| example.rst | 512B   | 文本文件 |
| data.csv    | 2KB    | 数据文件 |
''')
```

```
# 界面布局
ui.page_title("reST 文件阅读器")

# 渲染本地文件
rst = ui.restructured_text(read_rst_file('example.rst')).classes('p-4')

# 刷新按钮
ui.button('刷新文件', on_click=lambda:
rst.set_content(read_rst_file('example.rst'))).classes('mt-2')

ui.run()
```

## 2. 技术文档导航（结合标签页 + 锚点）

```
from nicegui import ui

# 文档内容（分章节）
docs = {
    '简介': '''
.. _intro:

简介
====
`ui.restructured_text` 是 NiceGUI 中用于渲染 reST 文档的组件，支持：

- 标准 reST 语法
- 代码块高亮
- 表格、公式、图片
- 动态更新与绑定
''',
    '快速开始': '''
.. _quickstart:

快速开始
=====
1. 安装依赖
.. code-block:: bash

    pip install nicegui docutils sphinx

2. 基础示例
.. code-block:: python

    from nicegui import ui
    ui.restructured_text('''# Hello reST!''')
    ui.run()
''',
    '高级功能': '''
.. _advanced:

高级功能
=====
```

### 1. 动态更新内容

参考「动态更新内容」章节。

### 2. 样式自定义

参考「样式自定义」章节。

### 3. 配置调整

通过 `settings` 参数配置解析器行为。

```
'''
}

# 界面布局
with ui.tabs() as tabs:
    ui.tab('简介')
    ui.tab('快速开始')
    ui.tab('高级功能')

with ui.tab_panels(tabs, value='简介'):
    for tab_name, content in docs.items():
        with ui.tab_panel(tab_name):
            # 渲染 reST 内容 + 锚点跳转链接
            ui.restructured_text(content).classes('p-4')
            # 锚点跳转链接
            with ui.row().classes('mt-4'):
                ui.link('跳至简介', '#intro').classes('mr-4')
                ui.link('跳至快速开始', '#quickstart').classes('mr-4')
                ui.link('跳至高级功能', '#advanced')

ui.run()
```

## 总结

`ui.restructured_text` 是 NiceGUI 中专注于 reST 文档渲染的核心组件，核心优势在于：

- 语法兼容性强**：完整支持 reST 标准语法及 `docutils` 扩展，适配技术文档常用元素；
- 易用性高**：内置代码高亮、表格解析、公式渲染等功能，无需额外配置；
- 灵活性强**：支持动态更新、数据绑定、样式自定义，可结合其他组件实现复杂交互；
- 场景适配**：天然适合技术文档、API 说明、学术文档等场景，是 reST 格式内容在 Web 界面中展示的最优选择。

使用时需注意 reST 语法规范和依赖安装，尤其是公式渲染和代码高亮功能。通过合理结合标签页、锚点、动态更新等功能，可快速构建专业的技术文档系统。